SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

1. Introduction

1.1. Purpose:

This System Analysis Documentation details the technical scope and architecture design decisions behind the development of AgroTech: Strategic Farmer Advisor. It serves as a blueprint for the development and QA teams to understand the end-to-end data flow, microservice integrations, and AI implementation required to empower Malaysian smallholder farmers.

So, the key element of system that has been covered are as followed:

1. ***Architecture**:*

AgroTech operates on a Client-Server Architecture delivered via a Web-App optimized for low-bandwidth rural areas. The system uses a Streamlit frontend for user interaction, a backend routing layer to siphon real-time environmental data, and integrates Gemini 3.1 Flash as the core "Strategic Decision Layer".

2. ***Data Flows:***

The flow begins when the user inputs unstructured data or selects their location, crop, and soil type via the UI. The backend siphons relevant localized weather and global CPO market data, combining it with the user's input. This context is sent to the Gemini API, which processes the variables and returns a structured "Situation Report" and strategic actionable advice back to the user's screen.

3. ***Model Process:***

The system utilizes a Multi-step Agentic Prompting workflow:

Step 1: Analyze unstructured user inputs (crop type and location-based environmental factors).

Step 2: Cross-reference user data with the specified agricultural context (e.g., Clay vs. Sandy soil requirements).

Step 3: Perform a trade-off analysis between immediate harvesting needs and long-term soil/crop health based on current trends.

Step 4: Translate the technical analysis into simple, actionable instructions in either Malay or English that the farmer can easily execute.

4. Role of Reference:

This document is a reference for all developers and testers to enable the overall high-level architecture of this project. It establishes clear API usage contracts for the development team and defines the boundary parameters (like the 1,500 token limit) and test cases for the QA team.

1.2. ***Background****:*

Malaysian smallholders, specifically in the palm oil sector and other crops like paddy and chili, rely heavily on traditional, manual methods for farm management and harvesting. Decisions based on visual intuition rather than data contribute to a 15-25% yield gap. AgroTech acts as a "Strategic Decision Layer" to process multi-modal data and provide accessible, data-driven actionable insights, shifting farmers from reactive laborers to proactive managers.

1.3. Target Stakeholder

Stakeholders	***Roles***	***Expectations***
Independent Smallholders	*Input farm details (location, crop type, soil) to receive localized strategic advice.*	*Simple, low-bandwidth Web-App experience providing easy-to-understand advice in English or Malay.*
Cooperation Managers	*Oversee multiple smallholder plots.*	*Receive clear situational reports to plan labor and expenses.*
MPOB Extension Officers	*Provide advisory services to communities.*	*Use the tool to validate field observations and scale their advisory reach.*
Development Team	*Build and maintain the platform.*	*Clear API integration strategies (Gemini, Open Weather) and modular codebase.*
QA Team	*Validating system behaviour.*	*Clear test cases ensuring 100% core logic pass rate and <800ms API response time.*

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	*Web-Application optimized for low-bandwidth rural areas.*
Architecture	*Client-Server Model with External API integrations.*

AgroTech is structured as a centralized web application. The frontend UI collects unstructured user inputs (location, crop type, soil). A backend routing layer siphons external weather and market data , passing the combined context to the Gemini API service layer.

2.1.2 LLM as Service Layer

AgroTech leverages Gemini 3.1 Flash as the core responding engine.

- *Prompt Construction: Uses a Multi-step Agentic Prompting approach. Step 1 analyzes inputs, Step 2 cross-references agricultural context, Step 3 performs trade-off analysis, and Step 4 translates the output into simple actionable instructions.*
- *Token & Input Limitations: The system enforces a maximum input size of 1,500 tokens per analysis session. The "Crop" field is hard-limited to 50 characters via the Streamlit text_input component to prevent prompt injection and payload errors.*
- *Validation Layer: A Rule-Based Validation Layer grounds the AI to prevent hallucinated market/weather data.*
-

2.1.3 Dependency Diagram

a. Prompt Construction & GLM API Integration: The Python Backend receives user inputs (Location, Crop, Soil) from the Streamlit UI. It simultaneously siphons localized weather data from the Open Weather API and market prices from the Local Cache. These inputs are injected into a Multi-step Agentic Prompt template.
b. Context Window: The context window includes: Unstructured farmer input (e.g., notes/images), environmental variables (Weather, Soil Type), and global CPO market trends.

c. Parsing Responses: The backend receives the Gemini 3.1 Flash response. A Rule-Based Validation Layer parses the JSON/Markdown response to ensure it passes basic agricultural constraints (e.g., yield limits). Once validated, it is passed to the Streamlit UI to be rendered bilingually.

d. Token Enforcement: Before the prompt is sent, the backend checks token length. If it exceeds 1,500 tokens, the system truncates the text input (hard limited at 50 characters for the 'Crop' field) to prevent API payload errors.

e. Major API Calls:

- *Streamlit UI calls Python Backend (User payload).*
- *Python Backend calls Open Weather API (Weather context).*
- *Python Backend calls Gemini 3.1 Flash API (Inference Engine).*

f. Component Interactions: The Backend communicates with a Local PostgreSQL Database/Cache to retrieve CPO prices, mitigating redundant GLM calls and rate limits.

2.2.2. Sequence Diagram

Customer Journey: Strategic Advice Request Flow

1. *User (Farmer): Opens the Streamlit Web-App and inputs Farm Details (Location: Alor Setar, Crop: Paddy, Soil: Clay).*
2. *Streamlit Frontend: Sends the data payload to the Python Backend.*
3. *Python Backend: Queries the Open Weather API for the 7-day forecast in Alor Setar.*
4. *Python Backend: Queries the Local Database/Cache for current Paddy/CPO market prices to avoid fetching redundant market data.*
5. *Python Backend: Combines user input, weather data, and market data into the Agentic Prompt and sends it to the Gemini 3.1 Flash API.*
6. *Gemini API: Performs "trade-off analysis" and returns a structured Situation Report and Actionable Advice.*
7. *Python Backend: Runs the "Rule-Based Validation" check to ensure no hallucinated financial advice.*
8. *Streamlit Frontend: Displays the localized, bilingual strategic advice to the user in under 800ms*

2.2. Technological Stack

2.2.1. Frontend: Streamlit (Ensures a lightweight, responsive UI for low-bandwidth areas).

2.2.2. Backend: Python (with PyTest for testing the Weather and Price Siphon functions).

2.2.3. Database/Storage: Local MetaData Caching (to store CPO prices and prevent redundant LLM queries) & SQL for dummy crop price history.

2.2.4. APIs: Gemini API, Open Weather API (mocked during testing).

2.3. Key Data Flows

2.3.1 Data Flow Diagram (DFD)

- *External Entities: Smallholders, Farm Managers, MPOB Officers*
- *Process 1 (Input Collection): The Streamlit UI collects unstructured text and environmental variables from the user.*
- *Process 2 (Context Siphoning): The backend actively fetches real-time external data (Open Weather API) and internal stored data (Market Price Cache).*
- *Process 3 (AI Inference): All collected context routes to the Gemini 3.1 Flash API for processing and translation.*
- *Process 4 (Validation & Output): The resulting advice is checked against safety rules and delivered to the user's dashboard.*
- *Data Stores: Local_Market_Cache (stores global CPO prices for 24h to reduce token costs), User_Session_Logs (for saving past advice).*
-

2.3.2. Normalized Database Schema

Here is the logical data model (ERD) designed to 3NF to ensure data integrity for AgroTech:

1. Table: Users
    - UserID (Primary Key)
    - Role (e.g., Smallholder, Cooperation Manager)
    - Preferred_Language (English / Malay)
2. Table: Farms
    - FarmID (Primary Key)
    - UserID (Foreign Key - linking user to their farms)
    - Location (e.g., Alor Setar)
    - Crop_Type (e.g., Paddy, Palm Oil)
    - Soil_Type (e.g., Clay, Sandy)
3. Table: MarketData_Cache
    - CacheID (Primary Key)
    - Crop_Type (e.g., Palm Oil, Paddy, Chili)
    - Current_Price (Float)
    - Last_Updated (Timestamp - ensures data is refreshed to prevent stale prices)
4. Table: Advice_Sessions (Junction/Log Table)
    - SessionID (Primary Key)
    - FarmID (Foreign Key)
    - Timestamp
    - Risk_Factor_Identified (e.g., "Heavy Rain")
    - Tokens_Used (Integer - tracked for budget threshold.

3. Functional Requirements & Scope

3.1. Minimum Viable Product:

#	Features	Description
1	Context-Aware Siphoning	System fetches localized weather and market trends based on user input (e.g., Alor Setar, Paddy).
2	Bilingual Command Center	Converts technical data into simple, actionable strategic advice in English or Bahasa Melayu.
3	Situation Report Engine	Identifies critical risk factors (e.g., Heavy Rain, Clay Soil) and suggests immediate actions.

3.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system. [e.g. table in below]

Quality (e.g.)	***Requirements (e.g.)***	***Implementation (e.g)***
Performance	*UI must feel responsive. API response time must be < 800ms.*	*Implement caching for market data so previous results are stored locally.*
Cost Efficiency	*LLM cost per user session must remain manageable (estimated RM0.15 to RM0.25).*	*Max input size restricted to 1,500 tokens; local metadata caching prevents redundant GLM querying.*
Reliability	*Zero critical bugs (P0/P1) in production; core logic must have a 100% pass rate.*	*Unit tests via PyTest isolate external dependencies using mocking to avoid API rate limits.*

3.3. Out of Scope / Future Enhancements

- *E-commerce capabilities (direct selling of crops or payment gateways).*
- *Long-term historical climate trend analysis.*
- *Automated execution of tasks via harvesting machinery (system acts only as an advisor).*

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. ***Validation Safety Net****: A "human-in-the-loop" basic validation mechanism requires the farmer to review the AI's situation report before actionable execution, ensuring data integrity.*

4.1.2. ***Fallback Strategy****: If ambiguous inputs are detected, the system triggers a Graceful Error State to ask the user to re-verify inputs rather than hallucinating.*

4.1.3. Priority Matrix:

Priority Level	***Feature / Component***	***Impact***	***Effort***	***Justification for MVP***
Must Have	***Gemini API Integration & Multi-Step Prompting:*** *Core logic for generating localized advice based on soil/crop/weather.*	*High*	*High*	*This is the core "Decision Intelligence" value proposition of AgroTech. Without this, the app fails to function.*
Must Have	***Streamlit UI (Bilingual):*** *Low-bandwidth frontend supporting English and Bahasa Melayu.*	*High*	*Low*	*This is the core "Decision Intelligence" value proposition of AgroTech. Without this, the app fails to function.*
Must Have	***Weather API Siphoning:*** *Fetching real-time local weather.*	*High*	*Medium*	*This is the core "Decision Intelligence" value proposition of AgroTech. Without this, the app fails to function.*

Should Have	***Local Market Price Cache:*** *Storing CPO/crop prices locally.*	*High*	*Medium*	*Highly recommended to minimize Gemini API token usage and latency.*
Could Have	***Session History Logs:*** *Saving past advice for users to review later.*	*Medium*	*Medium*	*Adds value to the user experience but is not strictly necessary to prove the core concept to judges.*
Won't Have (MVP)	***E-Commerce/Payment Gateway:*** *Selling crops directly via the app.*	*Low*	*High*	*Outside the scope of a "Strategic Advisor" tool; risks distracting from the AI implementation.*

4.2. Monitoring:

To ensure the system remains reliable, cost-effective, and safe during testing and eventual live deployment, the following monitoring strategies will be implemented:

1. ***API Usage & Cost Monitoring:***
   - ***Token Tracking:*** *The backend will log the number of tokens sent to and received from the Gemini 3.1 Flash API per session. This ensures the system stays within the estimated cost threshold (RM0.15 - RM0.25 per session) and respects the 1,500 token limit.*
   - ***Rate Limit Alerts:*** *Monitoring external calls to the Open Weather API and Gemini API. If the system approaches the free-tier rate limits, the backend will gracefully serve cached data or warn the user to try again later, rather than crashing outright.*
2. ***Performance & Latency Tracking:***
   - ***Response Time Logging:*** *Python's logging library will track the round-trip time of user requests. The QA goal is < 800ms. Any request exceeding 1.5 seconds will be flagged in the logs to investigate potential bottlenecks in the API siphoning or prompt processing.*

3. ***AI Safety & Hallucination Monitoring:***
    - ***Safety Filter Triggers:*** *As defined in the QATD, the system relies on Gemini's built-in safety settings (Hate/Harassment/Harmful blocked at "Medium and Above"). The system will log whenever a prompt is rejected by the API (e.g., adversarial inputs like "How to destroy a competitor's farm") to monitor for abuse.*
    - ***Fallback State Tracking:*** *The system will track how often the AI triggers a "Fallback State" (asking the user for clarification due to ambiguous inputs). A high frequency of fallbacks will indicate to the team that the Streamlit UI needs stricter input validation or dropdown menus instead of free-text fields.*
4. ***System Health (Uptime):***
    - *Basic health-check endpoints will be established in the backend to ensure the Streamlit app and the connection to the Local PostgreSQL database are active.*

4.3. Assumptions

- ***Connectivity:*** *Users have an active internet connection to access the Gemini cloud.*
- ***Human Approval:*** *The system serves as decision-support; users must manually execute financial or physical actions.*

4.4. External Dependencies

Tools	*Purpose*	*Risks*
Gemini 3.1 Flash API	*Core LLM reasoning engine for multi-step prompting and structured output generation.*	*High - Free keys stop working due to rate limits. Mitigated via local caching for a few hours.*
Open Weather API	*Siphons localized weather data to provide context to the LLM.*	*Medium - API downtime. Mitigated by isolating and mocking during CI/CD checks.*

5. Project Management & Team Contributions

5.1. Project Timeline

Given the preliminary round's duration of approximately 10 days, the team is utilizing a fast-paced Agile sprint methodology to ensure the Minimum Viable Product (MVP) is delivered on time without compromising core features.

1. ***Day 1-2: Project Planning & Architecture Design***
    - *Finalize the PRD, SAD, and QATD.*
    - *Map out the high-level architecture, database schemas, and multi-step prompt design.*
2. ***Day 3-4: UI/UX Prototyping & Backend Setup***
    - *Develop the Streamlit Web-App interface.*
    - *Set up the Python backend routing and Local Market Price caching mechanisms.*
3. ***Day 5-6: Core AI Integration & External Siphoning***
    - *Integrate the Gemini 3.1 Flash API and fine-tune the agentic prompt context window.*
    - *Connect and siphon data from the Open Weather API (or build mock endpoints for testing).*
4. ***Day 7-8: Quality Assurance & Bug Fixing***
    - *Execute all test cases outlined in the QATD (Token limits, API latency, Edge/Adversarial prompts).*
    - *Refine the Rule-Based Validation safety net based on AI hallucination checks.*
5. ***Day 9: Final Review & Pitch Alignment***
    - *Freeze code development.*
    - *Ensure the system perfectly aligns with the slide deck, demo flow, and strategic objectives.*
6. ***Day 10: Final Deployment & Hackathon Submission***
    - *Record the demo and submit all required documentation (Canva slides, PRD, SAD, QATD).*

5.2. Team Members Role

- ***Name:*** *Muhammad Farish Chai - Project Leader*
- ***Name:*** *Muhammad Amirul Hakimi - Creative Designs*
- ***Name:*** *Hariz Arfan - Documentation*
- ***Name:*** *Muhamad Danish - Pitching Editor*
- ***Name:*** *Harith Danish - Lead Developer*

5.3. Recommendations (Scalability & Future Proofing)

While AgroTech currently relies on a lightweight stack suitable for a 10-day hackathon MVP, the following architectural recommendations are necessary for scaling the product to support Malaysia's 500,000+ smallholder farmers in production:

1. ***Database & Cloud Scalability:*** *Transition from local metadata caching to a robust, cloud-native managed database (e.g., PostgreSQL on AWS or Google Cloud SQL). This will support high concurrent user sessions and allow for scalable historical data storage (e.g., crop yield tracking over multiple years).*
2. ***Direct MPOB API Integration:*** *Instead of relying on dummy historical price databases or basic web scraping, the system should officially partner with the Malaysian Palm Oil Board (MPOB) and local cooperatives to ingest real-time, highly localized market data via secure enterprise APIs.*
3. ***Multi-Modal Vision Capabilities:*** *To further reduce the reliance on manual text inputs, the system should be upgraded to utilize Gemini's multi-modal capabilities. Farmers could upload images of diseased crops or soil conditions, allowing the AI to automatically extract variables rather than requiring the farmer to type them out.*
4. ***Voice-to-Text Integration:*** *To make the application truly accessible to older or less tech-savvy farmers, integrating a Voice-to-Text model (handling Bahasa Melayu dialects) into the Streamlit UI would drastically improve the user experience.*

6. *References*

This project, developed by **Team Error 404**, adheres to the official submission guidelines of UMHackathon 2026 regarding the disclosure of external resources and artificial intelligence tools. The following resources were utilized during the developmental period:

- **Generative AI Reasoning Engine:** * **Google Gemini 3.1 Flash API:** Utilized as the central "Reasoning Service" to perform context-aware analysis of agricultural data and generate strategic instructions in both English and Malay.
    - **Developmental Assistance:** Large Language Models were used for code optimization, content generation, and technical documentation assistance.

- **External Data Sources & APIs:**
    - **OpenWeather API:** Integrated to fetch real-time environmental data (temperature, precipitation, and weather conditions) for specific Malaysian locations.
    - **Malaysian Agricultural Market Indices:** Publicly accessible price trends for Paddy, Chili, and Tomato were referenced to provide economic insights.

- **Software Frameworks & Libraries:**
    - **Streamlit:** Used as the primary cloud-native infrastructure for the user interface and frontend deployment.
    - **Python Libraries:** Standard libraries including google-generativeai and requests were used for API integration and data siphoning.

- **Compliance Statement:**
    - All external tools and datasets used fulfill the criteria of being publicly accessible and properly cited.
    - This submission is exclusive to UMHackathon 2026 and has not been entered into any other concurrent events.
    - No sensitive, identifiable personal information was used or shared within the datasets or problem statement responses.